RELATÓRIO TÉCNICO · ANÁLISE DE CÓDIGO

Inversão Gravimétrica 1D com Regularização por Variação Total

Documentação e análise do código Python para modelagem direta e inversão do relevo do embasamento a partir de anomalias gravimétricas.

Repositório	`tcc/codigo`
Escopo	Diretório `python/` — 16 módulos
Método	Quasi-Newton (BFGS) com amortecimento de Marquardt e regularização por Variação Total
Origem	Portado do código MATLAB em `TVGRAV1D/` e `matlab/`
Dependências	NumPy, Matplotlib
Data	3 de setembro de 2026

Sumário

1. Escopo e organização do projeto

Estrutura de diretórios · Inventário de módulos · Grafo de dependências

2. Fundamentação matemática

Problema direto · Parametrização · Funcional de Variação Total · Otimização

3. Modelagem direta — módulos da raiz

`grav2DPrism2.py` · `modGrav1d2.py` · `modGrav1dTOTAL.py` · `Mod2DGravim.py`

4. Pacote `tv1d` — núcleo numérico

Problema direto e derivadas · Funcional TV · Função objetivo · Modelo inicial

5. Pacote `tv1d` — otimização

`matinv.py` · `QNiterate.py` · `inv_1d.py`

6. Visualização — `plotter.py`

7. Fluxo de execução e parâmetros de controle

8. Resultado numérico de referência

9. Correspondência MATLAB → Python

10. Pontos de atenção e melhorias sugeridas

1. Escopo e organização do projeto

Este relatório documenta o código Python do projeto após a remoção de todo o material referente à inversão gravimétrica 2D. O que resta é um conjunto coeso e mínimo voltado exclusivamente ao problema unidimensional: estimar o relevo do embasamento de uma bacia sedimentar a partir de um perfil de anomalia gravimétrica.

1.1 Estrutura de diretórios

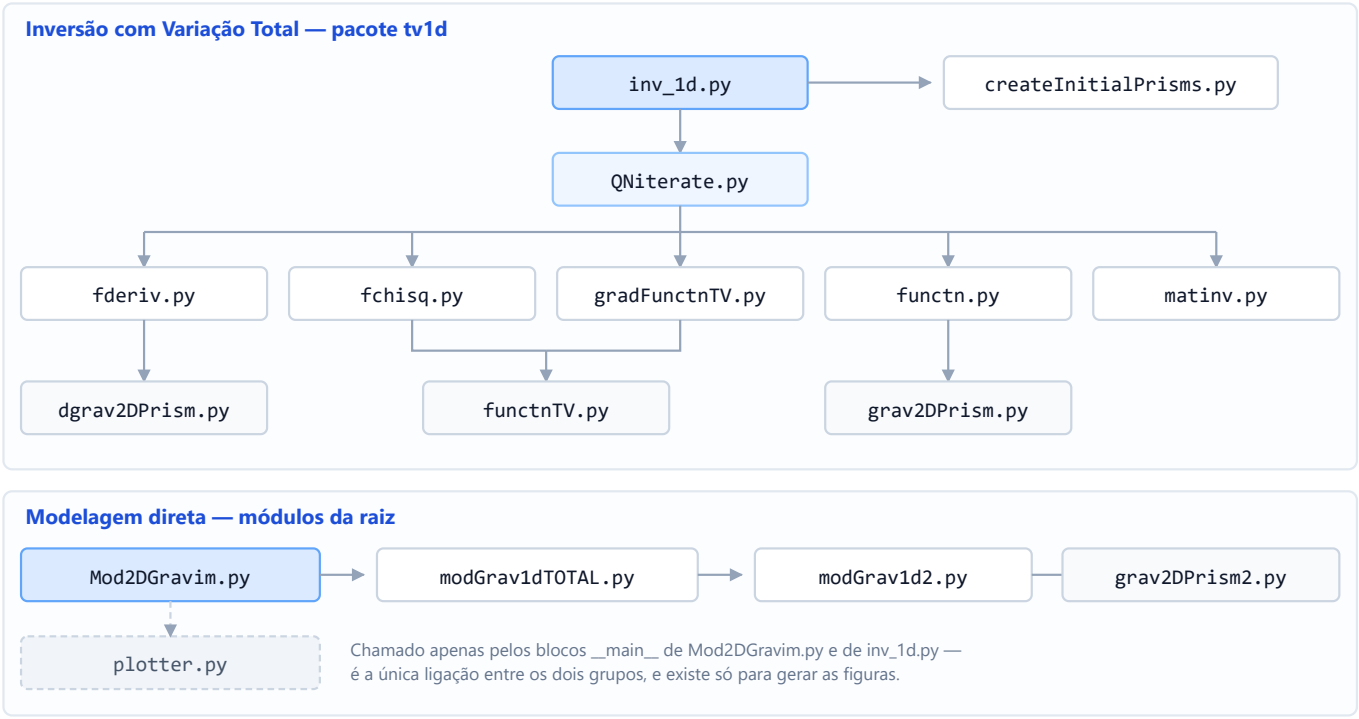
```
python/
├── grav2DPrism2.py      # anomalia de um prisma 2D (versão math)
├── modGrav1d2.py        # soma de prismas em um ponto de observação
├── modGrav1dTOTAL.py    # varre todos os pontos de observação
├── Mod2DGravim.py       # configuração do modelo + demonstração
├── plotter.py           # figuras (modelagem direta e inversão)
└── tv1d/                # pacote de inversão 1D com Variação Total
    ├── grav2DPrism.py       # anomalia de um prisma 2D (versão NumPy)
    ├── functn.py            # problema direto vetorizado sobre XOBS
    ├── dgrav2DPrism.py      # derivada analítica ∂g/∂h
    ├── fderiv.py            # uma linha da matriz jacobiana
    ├── functnTV.py          # funcional de Variação Total
    ├── gradFunctnTV.py      # gradiente numérico do funcional TV
    ├── fchisq.py            # função objetivo (ajuste + regularização)
    ├── createInitialPrisms.py # modelo inicial por aproximação de Bouguer
    ├── matinv.py            # inversão de matriz por Gauss-Jordan
    ├── QNiterate.py         # uma iteração Quasi-Newton (BFGS + Marquardt)
    └── inv_1d.py            # laço principal da inversão + demonstração
```

São dois blocos com responsabilidades distintas. Os quatro módulos da raiz formam a cadeia de **modelagem direta** herdada do código MATLAB em `matlab/`, usada pela inversão por Levenberg-Marquardt do relevo do embasamento. O pacote `tv1d/` é o porte do código MATLAB em `TVGRAV1D/` e contém a implementação completa da **inversão regularizada por Variação Total**, incluindo sua própria cópia do problema direto.

1.2 Inventário de módulos

Módulo	Papel	Entrada → saída
`grav2DPrism2.py`	Direto	Parâmetros de um prisma → contribuição escalar em mGal
`modGrav1d2.py`	Direto	Vetor de espessuras → anomalia em um ponto
`modGrav1dTOTAL.py`	Direto	Vetor de espessuras → anomalia em todo o perfil
`Mod2DGravim.py`	Configuração	Modelo → anomalia, com geometria fixa (60 prismas)
`plotter.py`	Visualização	Vetores → figuras Matplotlib de dois painéis
`tv1d/grav2DPrism.py`	Direto	Parâmetros de um prisma → contribuição escalar
`tv1d/functn.py`	Direto	Modelo P → resposta $F(P)$ em todo o perfil
`tv1d/dgrav2DPrism.py`	Derivada	Prisma → $\partial g/\partial h$ analítica
`tv1d/fderiv.py`	Derivada	Modelo, índice i → linha i da jacobiana
`tv1d/functnTV.py`	Regularização	Modelo → escalar TV(P)
`tv1d/gradFunctnTV.py`	Regularização	Modelo → ∇TV(P)
`tv1d/fchisq.py`	Objetivo	Dados, resposta, modelo → (objetivo, RMS, TV)
`tv1d/createInitialPrisms.py`	Inicialização	Dados observados → modelo inicial
`tv1d/matinv.py`	Álgebra linear	Matriz → inversa e indicador de singularidade
`tv1d/QNiterate.py`	Otimização	Estado da iteração → estado atualizado
`tv1d/inv_1d.py`	Orquestração	Dados observados → modelo final P

1.3 Grafo de dependências



As duas colunas são independentes: nenhum módulo da raiz importa `tv1d`, e nenhum módulo de `tv1d` importa a raiz — a única exceção é o bloco de demonstração de `inv_1d.py`, que insere o diretório-pai em `sys.path` para usar `plotter.py` (linhas tracejadas).

2. Fundamentação matemática

2.1 O problema direto

A anomalia gravimétrica vertical produzida por um prisma bidimensional de contraste de densidade constante — infinito na direção perpendicular ao perfil — é dada pela expressão analítica de Telford, Geldart & Sheriff (*Applied Geophysics*, p. 73, eq. 2.49c). Para um prisma de largura w, espessura h, topo em z_0 e centro horizontal x_0, medido em um ponto x_{obs}:

$$g_z = 2 \times 10^5 \, G\Delta\rho \, \left[x \ln(r_2 r_3 / r_1 r_4) + w \ln(r_4 / r_3) - h(\theta_4 - \theta_2) + z_0(\theta_3 - \theta_1) \right] \tag{1}$$

com $x = x_{obs} - x_0 + w/2$, e as distâncias $r_{1..4}$ e os ângulos $\theta_{1..4}$ medidos aos quatro vértices do prisma:

$$r_1 = \sqrt{(x^2 + z_0^2)} \quad r_2 = \sqrt{(x^2 + (h+z_0)^2)} \quad r_3 = \sqrt{((x-w)^2 + z_0^2)} \quad r_4 = \sqrt{((x-w)^2 + (h+z_0)^2)}$$

A constante 2×10^5 reúne o fator 2 da geometria bidimensional e a conversão de unidades. O código adota $G = 6{,}672 \times 10^{-8}$ (CGS) e converte o contraste de densidade de g/cm³ para as unidades internas multiplicando por 1000. Com distâncias em quilômetros o resultado sai em **miligal**; com distâncias em metros, em **microgal** — daí a divisão por 1000 controlada pelo parâmetro `unit`.

2.2 Parametrização do modelo

O relevo do embasamento é discretizado por n prismas justapostos de largura fixa, todos com topo na superfície ($z_0 = 0{,}0005$, um valor pequeno mas não nulo para evitar singularidade nos arco-tangentes). A incógnita é o vetor de **espessuras** $P = (P_1, \ldots, P_n)$, que corresponde diretamente à profundidade do embasamento sob cada prisma. A anomalia predita em um ponto é a soma das contribuições:

$$F_j(P) = \Sigma_{i=1}^{n} g_z(\Delta\rho, x_{c,i}, z_0, w, P_i, x_{obs,j}) \tag{2}$$

onde $x_{c,i} = x_{min} + (i-1)w + w/2$ é o centro do i-ésimo prisma. Note que o modelo é linear na geometria mas **não linear nos parâmetros**: a dependência de g_z em h envolve logaritmos e arco-tangentes.

2.3 O problema inverso e a Variação Total

A inversão minimiza um funcional que combina o desajuste aos dados com um termo estabilizador:

$$T(P) = \| d_{obs} - F(P) \|^2 + \alpha \, \mathrm{TV}(P) \tag{3}$$

O estabilizador de **Variação Total** é a norma L_1 das diferenças entre parâmetros adjacentes. Ao contrário do vínculo clássico de suavidade (norma L_2 das diferenças), a norma L_1 não penaliza saltos isolados de grande amplitude, o que permite recuperar **descontinuidades abruptas** — falhas, bordas de grabens — sem borrá-las. Como $|\cdot|$ não é diferenciável na origem, o código usa a aproximação suavizada de Acar & Vogel:

$$\mathrm{TV}(P) = \Sigma_{i=1}^{n-1} \sqrt{((P_{i+1} - P_i)^2 + \beta^2)} \tag{4}$$

com β pequeno (padrão 0,001). Quanto menor β, mais próxima a aproximação da norma L_1 exata — e mais mal-condicionado o problema. O parâmetro α controla o balanço entre ajustar os dados e produzir um modelo constante por partes.

2.4 Otimização

O gradiente do funcional (3) em relação ao modelo é

$$\nabla T(P) = -2\, J^{\top} (d_{\mathrm{obs}} - F(P)) + \alpha\, \nabla \mathrm{TV}(P) \tag{5}$$

onde $J_{ji} = \partial F_j / \partial P_i$ é a matriz jacobiana, calculada **analiticamente** em `dgrav2DPrism.py`. O passo é obtido resolvendo um sistema com a hessiana aproximada H_k, escalonada e amortecida à Marquardt:

$$P^{k+1} = P^k - (H_k + \lambda I)^{-1} \nabla T(P^k) \tag{6}$$

A hessiana é atualizada a cada iteração aceita pela fórmula **BFGS**, com $s_k = P^{k+1} - P^k$ e $y_k = \nabla T^{k+1} - \nabla T^k$:

$$H_{k+1} = H_k + y_k y_k^{\top} / (y_k^{\top} s_k) - H_k s_k s_k^{\top} H_k / (s_k^{\top} H_k s_k) \tag{7}$$

> **Por que Quasi-Newton**
>
> A hessiana exata do termo de Variação Total é cara e mal-condicionada. O BFGS a constrói implicitamente a partir de diferenças sucessivas de gradiente, exigindo apenas o gradiente analítico — barato, já que a jacobiana tem forma fechada. O amortecimento de Marquardt garante que cada passo seja de descida mesmo quando a aproximação de hessiana ainda é pobre (nas primeiras iterações, $H_0 = I$).

3. Modelagem direta — módulos da raiz

Cadeia de quatro funções que implementa a equação (2), portada quase linha a linha do MATLAB. Usa apenas a biblioteca padrão (`math`), sem NumPy.

grav2DPrism2.py — 36 linhas · biblioteca padrão

`grav2DPrism2(den, x0, z0, w, h, xobs)` Avalia a equação (1) para um único prisma e devolve um escalar.

Tradução direta da fórmula de Telford. O tratamento de `z0 == 0` é explícito: como `math.atan(x/z0)` dividiria por zero, o código devolve ±π/2 conforme o sinal de x. Na prática esse ramo nunca é exercitado, pois os chamadores sempre passam `ztop = 0.0005`.

```
def grav2DPrism2(den, x0, z0, w, h, xobs):
    CG = 6.672e-08
    cd = den * 1000.0
    x = xobs - x0 + w / 2.0
    r1 = math.sqrt(x**2 + z0**2)
    # ... r2, r3, r4, theta1..theta4 ...
    gz = 2 * (10**5) * CG * cd * (
        x * math.log((r2 * r3) / (r1 * r4)) + w * math.log(r4 / r3)
        - h * (theta4 - theta2) + z0 * (theta3 - theta1))
    return gz
```

`modGrav1d2.py` — 23 linhas

`modGrav1d2(P, XOBS, iobs, xminPrisms, prismWidth, prismDensity, unit=0)` — Soma a contribuição de todos os prismas em **um** ponto de observação `XOBS[iobs]`.

Percorre o vetor `P`, calcula o centro `xc = xminPrisms + i*prismWidth + prismWidth/2` de cada prisma e acumula. O topo é fixado em `ztop = 0.0005`.

> **Correção de unidade aplicada nesta revisão**
>
> Até esta revisão a função terminava com `functn = functn / 1000.0` *sem* verificar a unidade de distância. No MATLAB original (`matlab/modGrav1d2.m`) o teste `if(unit==1)` que envolvia essa linha está comentado, e o porte havia reproduzido o comportamento fielmente. Como `Mod2DGravim` trabalha em quilômetros, a equação (1) já devolve miligal e a divisão adicional deixava a saída 1000 vezes menor que a anomalia física.
>
> O parâmetro `unit` foi reintroduzido nas três funções da cadeia, com o mesmo contrato adotado em `tv1d/functn.py`: `0` = quilômetros, resultado em mGal (padrão); `1` = metros, com a conversão de µGal para mGal. Nenhuma chamada existente precisou mudar.

> **Os dados históricos estão na convenção antiga**
>
> A correção levanta a questão de em que unidade estão os dados de `matlab/gobsGI.mat`, usados por `InversaoRelevoGravimetriaML.m`. A verificação é direta: aplicando o operador ao modelo verdadeiro guardado em `modeloGI.mat` — espessuras de 0,3 a 1,85 km — e comparando com a anomalia observada,
>
`gobsGI.mat` (amplitude máxima)	`-0,018712`
`F(modeloGI, unit=1)` — convenção antiga	`-0,018325`
`F(modeloGI, unit=0)` — mGal, padrão atual	`-18,325`
RMS do resíduo com `unit=1`	$3{,}4 \times 10^{-4}$
>
> O ajuste com `unit=1` é de 0,03 % — **o arquivo `gobsGI.mat` foi gerado com o operador dividido por 1000**. O pipeline MATLAB é, portanto, internamente consistente: como dado observado e problema direto compartilhavam o mesmo fator, as espessuras recuperadas em quilômetros sempre estiveram corretas. O que estava errado era apenas a unidade declarada da anomalia, não o resultado da inversão.
>
> Consequência prática: ao usar `gobsGI.mat` a partir do Python, passe `unit=1` — ou multiplique o dado por 1000 e use o padrão, que é o caminho recomendado por deixar tudo em mGal.

`modGrav1dTOTAL.py` — 15 linhas

`modGrav1dTOTAL(P, XOBS, xminPrisms, prismWidth, prismDensity, unit=0)` — Repete `modGrav1d2` para cada ponto do perfil e devolve a lista completa.

Laço externo sobre as observações. Custo $\mathcal{O}(n_{\text{obs}} \times n_{\text{prismas}})$ chamadas escalares — 3.600 avaliações na configuração padrão.

`Mod2DGravim.py` 38 linhas · ponto de entrada executável

`Mod2DGravim(m, unit=0)` Encapsula a geometria fixa do experimento e devolve a anomalia do modelo `m`.

Fixa 60 prismas de 1 km de largura ao longo de $x \in [0, 60]$, contraste de densidade −0,25 g/cm³ e 60 pontos de observação em $x = 1 \ldots 60$. É a assinatura que a inversão MATLAB por Levenberg-Marquardt espera: uma função de um único argumento, o modelo.

O bloco `__main__` monta um modelo sintético em degraus — `[0]*10 + [5]*10 + [10]*20 + [5]*10 + [0]*10` — calcula a anomalia e a exibe com `plot_model_and_anomaly`.

> **Geometria unificada nesta revisão**
>
> As constantes `nprisms`, `nobs` e `xmaxPrisms` eram declaradas mas nunca usadas: a largura do prisma estava fixada em `1` e o eixo de observação escrito à mão como `range(1, 61)`, de modo que o número real de prismas vinha implicitamente de `len(m)`. Um modelo de comprimento diferente de 60 era aceito em silêncio, com a geometria deixando de corresponder ao declarado.
>
> Agora as constantes governam a geometria — `prismWidth = (xmaxPrisms - xminPrisms) / nprisms` e `xobs = range(1, nobs + 1)`, o mesmo padrão de `inv_1d` — e um modelo com comprimento incompatível levanta `ValueError` em vez de produzir um resultado silenciosamente errado. A saída numérica é idêntica à anterior para o caso de 60 prismas.

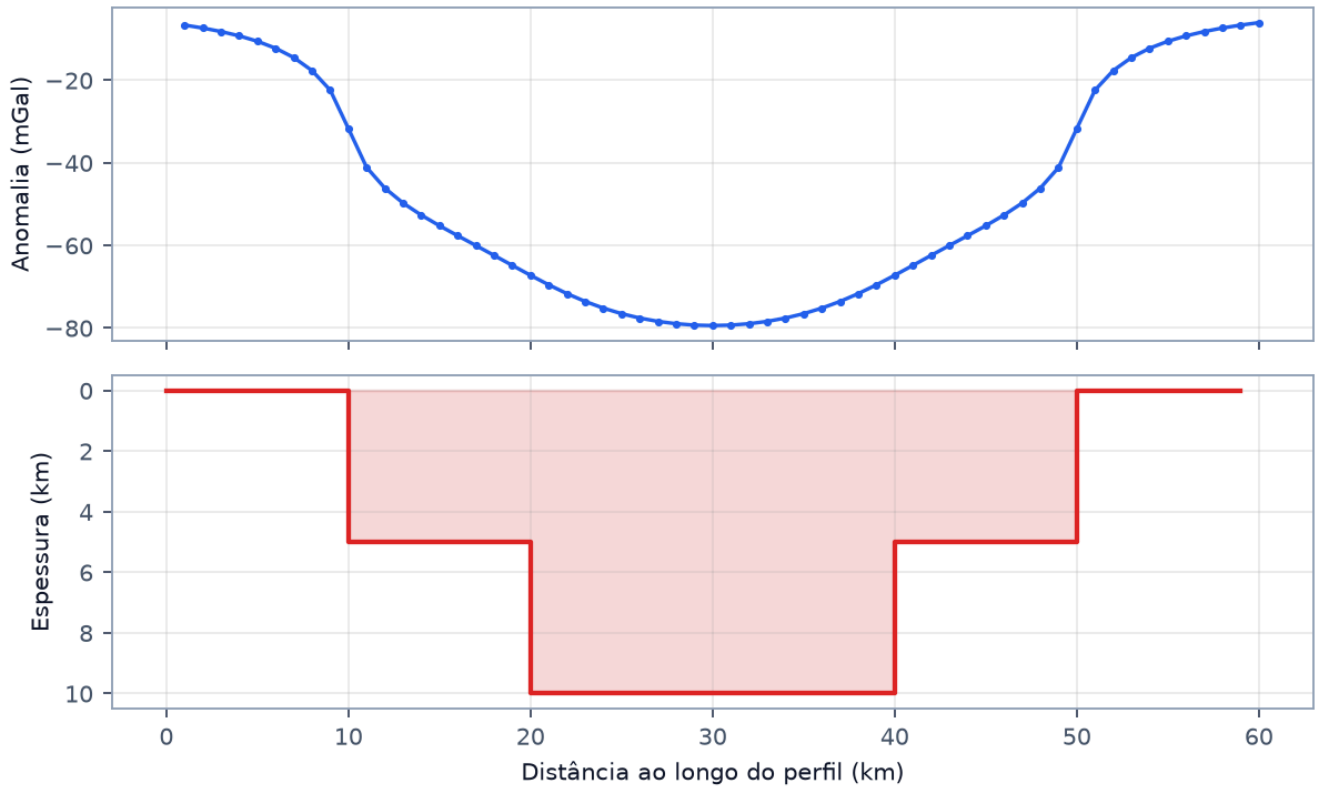


Figura 1. Saída de `python Mod2DGravim.py` *após* a correção de unidade. Painel superior: anomalia calculada nos 60 pontos de observação. Painel inferior: modelo sintético de espessuras, com o eixo de profundidade invertido. O contraste de densidade negativo produz o mínimo de anomalia sobre a região mais espessa, como esperado. A **amplitude** agora é fisicamente coerente: −79,4 mGal sobre o bloco de 10 km com Δρ = −0,25 g/cm³, contra os −104,8 mGal que a placa de Bouguer infinita ($2\pi G \Delta\rho h$) preveria — a diferença é o efeito de borda esperado para um bloco de apenas 20 km de largura. Antes da correção esta mesma curva marcava −0,079 mGal.

4. Pacote `tv1d` — núcleo numérico

Os módulos desta seção implementam tudo o que a otimização precisa avaliar: a resposta do modelo, suas derivadas, o funcional regularizador e a função objetivo. Todos usam NumPy e adotam o mesmo padrão de importação — tentativa relativa (`from .x import x`) com recuo para absoluta — de modo que funcionam tanto como pacote quanto como scripts soltos.

4.1 Problema direto e derivadas

`tv1d/grav2DPrism.py` — 28 linhas · NumPy

Mesma equação (1) de `grav2DPrism2.py`, com duas diferenças relevantes: usa `np.arctan2(x, z0)` em vez de `math.atan(x/z0)` — numericamente estável e correto em todos os quadrantes — e opera com `np.sqrt`/`np.log`, aceitando entradas vetoriais. É esta versão que a inversão utiliza.

`tv1d/functn.py` — 25 linhas

`functn(P, XOBS, xminPrisms, prismWidth, prismDensity, dcType, dcBeta, unit)` — Implementa a equação (2): devolve `np.ndarray` de comprimento `len(XOBS)`.

Laço externo sobre prismas, interno sobre observações — o inverso de `modGrav1dTOTAL`, mas equivalente por comutatividade da soma. A divisão por 1000 é aplicada apenas quando `unit == 1` (distâncias em metros), como no MATLAB original de `TVGRAV1D/`.

`dcType` seleciona o modelo de contraste de densidade: `0` = constante (implementado), `1` = variação hiperbólica com a profundidade, que levanta `NotImplementedError`. O parâmetro `dcBeta` só teria efeito nesse segundo caso.

`tv1d/dgrav2DPrism.py` — 27 linhas

`dgrav2DPrism(den, x0, z0, w, h, xobs, unit)` — Derivada analítica $\partial g_z/\partial h$ da equação (1) em relação à espessura do prisma.

Diferenciando (1) e agrupando termos, a derivada se decompõe em três parcelas:

$$p_1 = 2(z_0+h)\,[\,1/((z_0+h)^2+x^2) - 1/((z_0+h)^2+(x-w)^2)\,]$$
$$p_2 = 2(z_0+h)\,/\,((z_0+h)^2+(x-w)^2)$$
$$p_3 = \theta_1-\theta_2 + \text{termos de correção dos arco-tangentes}$$
$$\partial g/\partial h = 2\times 10^5\, G\Delta\rho\,[\,(x/2)p_1 + (w/2)p_2 - p_3\,]$$

Diferentemente da função direta, aqui $x = x_{obs} - x_0$, **sem** o deslocamento de $w/2$ — a origem local está na borda esquerda do prisma. Ter a derivada em forma fechada é o que torna o método viável: elimina n avaliações extras do problema direto por linha da jacobiana.

tv1d/fderiv.py 21 linhas

`fderiv(i, P, XOBS, xminPrisms, prismWidth, prismDensity, dcType, dcBeta, unit)`	Monta a linha i da jacobiana: vetor de comprimento `len(P)` com $\partial F_i/\partial P_k$.

A jacobiana nunca é montada por inteiro em memória: `QNiterate` percorre as observações e acumula o produto $J^{\top} r$ linha a linha, o que mantém o uso de memória em $O(n)$.

4.2 Regularização por Variação Total

tv1d/functnTV.py 29 linhas

Avalia a equação (4) — soma de $n-1$ termos. Implementação direta em laço Python; para os tamanhos usados (dezenas a poucas centenas de prismas) o custo é desprezível diante do problema direto.

```
TV = 0.0
for i in range(len(P) - 1):
    dp = P[i + 1] - P[i]
    TV += np.sqrt(dp ** 2 + beta ** 2)
```

tv1d/gradFunctnTV.py 24 linhas

`gradFunctnTV(P, beta)` Gradiente de TV por **diferenças finitas centradas**.

Para cada parâmetro, perturba `P[i]` de $\pm dp$ e aplica $(\mathrm{TV}_{+} - \mathrm{TV}_{-})/(2\,dp)$. O passo é proporcional ao próprio valor, `dp = 0.01*P[i]`, com piso `dp = 0.001` para não degenerar quando `P[i]` se aproxima de zero.

> **Gradiente analítico disponível**
> A derivada de (4) tem forma fechada — $\partial \mathrm{TV}/\partial P_i = d_{i-1}/\sqrt{(d_{i-1}{}^2+\beta^2)} - d_i/\sqrt{(d_i^2+\beta^2)}$, com $d_i = P_{i+1} - P_i$. Substituir as $2n$ avaliações numéricas por essa expressão reduziria o custo do termo de regularização de $O(n^2)$ para $O(n)$ e eliminaria o erro de truncamento. Optou-se por manter a versão numérica por fidelidade ao código MATLAB de referência.

tv1d/fchisq.py 43 linhas

`fchisq(GOBS, YFIT, P, alpha, betaTV)` Devolve a tripla (`chisqR, chisqA, tvValue`).

Saída	Definição	Uso
`chisqR`	$\|\|d_{obs} - F\|\|^2 + \alpha\,TV$	Valor minimizado; decide aceitação do passo
`chisqA`	$\sqrt{(\|\|d_{obs} - F\|\|^2 / n_{obs})}$	RMS em mGal; métrica de diagnóstico reportada
`tvValue`	TV(P), equação (4)	Acompanha a evolução da rugosidade do modelo

A separação é importante: `chisqR` mistura unidades (mGal² e km) e não tem interpretação física direta, enquanto `chisqA` é comparável ao nível de ruído dos dados e é o critério que o intérprete usa para julgar o ajuste.

4.3 Modelo inicial

tv1d/createInitialPrisms.py 38 linhas

`createInitialPrisms(nobs, P, udist, anomObs, prismWidth, prismDensity)` Estimativa inicial das espessuras pela **aproximação de Bouguer**.

A anomalia de uma placa infinita de espessura h e contraste $\Delta\rho$ é $g = 2\pi G \Delta\rho h$. Invertendo e usando a constante em unidades práticas:

$$P_k = u_{fact} \cdot \langle g \rangle_k \,/\, (41\{,\}9\,\Delta\rho) \tag{8}$$

O termo $\langle g \rangle_k$ é a anomalia observada na vizinhança do prisma k, ponderada por $1/d^2$. O raio de busca começa em `prismWidth/4` e cresce nesse mesmo incremento até capturar ao menos uma observação válida, o que torna o procedimento robusto a perfis com amostragem irregular ou lacunas — observações marcadas com `-9999.00` são descartadas.

O fator `ufact` vale 1 para distâncias em quilômetros (`udist = 0`) e 10^3 para metros. O somatório `+ 0.0001` nos denominadores evita divisão por zero quando a observação coincide com o centro do prisma ou quando o contraste é nulo.

5. Pacote `tv1d` — otimização

`tv1d/matinv.py` 75 linhas

`matinv(ARRAY, NORDER) → (A_inv, SFLAG)` Inversão de matriz por eliminação de **Gauss-Jordan com pivotamento total**.

Porte literal da rotina clássica de Bevington (via MATLAB, via Fortran). Procura o pivô de maior módulo em toda a submatriz restante, registra as permutações em `IK` e `JK`, elimina, e ao final desfaz as trocas na ordem inversa. `SFLAG = 1` sinaliza matriz singular (pivô nulo).

O próprio código anota que `np.linalg.inv` seria o equivalente moderno. A permanência da rotina manual é deliberada: garante reprodutibilidade bit a bit em relação ao MATLAB, útil na validação do porte. Custo $O(n^3)$ em laços Python puros — é o gargalo dominante da inversão.

tv1d/QNiterate.py — 152 linhas · núcleo do método

`QNiterate(nprisms, P, anomObs, YFIT, …, flambda, it, unit, state)`	Executa **uma** iteração Quasi-Newton e devolve `(chisqr, chisqA, tvValue, YFIT, P, flambda, state)`.

Gerenciamento de estado

No MATLAB, a hessiana aproximada `Hk` e o gradiente anterior `bbeta` são variáveis `persistent`. O porte substitui esse mecanismo por um dicionário `state` explícito, criado pelo chamador:

```
state = {'Hk': np.eye(nprisms), 'bbeta': np.zeros(nprisms)}
```

A troca é uma melhoria de projeto: torna a função reentrante, permite rodar várias inversões em paralelo no mesmo processo e deixa o estado inspecionável — o que `persistent` não permitia.

Anatomia de uma iteração

1. **Gradiente.** Na primeira iteração, calcula ∇T pela equação (5) percorrendo as observações e acumulando `(GOBS[i] - YFIT[i]) * DERIV`. Nas demais, reaproveita o gradiente já calculado ao fim da iteração anterior — economiza uma varredura completa da jacobiana.
2. **Escalonamento.** Monta $\tilde{H}_{jk} = H_{jk} / \sqrt{|H_{jj} H_{kk}|}$, normalizando a diagonal para 1. Sem isso, o amortecimento λI teria efeito muito diferente entre parâmetros de escalas distintas.
3. **Amortecimento.** Soma λ à diagonal, inverte com `matinv` e calcula o passo pela equação (6). Espessuras negativas são truncadas em `0.01` — vínculo de positividade que reflete o fato de o embasamento não poder subir acima da superfície.
4. **Aceitação.** Avalia a função objetivo no modelo candidato. Se não houve redução, multiplica λ por 10 e repete; a partir de $\lambda > 10^8$ desiste e devolve `flambda = -1`, sinal de parada para o laço externo.
5. **Atualização BFGS.** Aceito o passo, recalcula o gradiente no novo modelo, forma s_k e y_k e aplica a equação (7). Um $\varepsilon = 10^{-6}$ nos denominadores protege contra passos degenerados.

```
# atualização BFGS da hessiana aproximada
SK = B - P
YK = bbeta - beta
XM2 = np.outer(YK, YK);  E1 = np.dot(YK, SK)
V1  = Hk @ SK
XM1 = np.outer(V1, V1);  E2 = np.dot(SK, V1)
Hk = Hk + XM2 / (E1 + eps) - XM1 / (E2 + eps)
```

> **O amortecimento reinicia a cada iteração**
>
> O `flambda` devolvido é o valor recebido, não o `lambda_` aumentado dentro do laço de Marquardt. Cada iteração externa recomeça, portanto, do λ inicial. É o comportamento do MATLAB de referência e favorece passos grandes, mas difere do Levenberg-Marquardt clássico, que também *reduz* λ após sucessos e carrega o valor adiante.

A variável `idfp` (0 = BFGS, 1 = DFP), atribuída mas nunca lida — resquício de uma versão do MATLAB que alternava entre as duas fórmulas de atualização —, foi removida nesta revisão. A rotina implementa exclusivamente BFGS.

tv1d/inv_1d.py 213 linhas · ponto de entrada executável

`inv_1d(anomObs, maxIter, flambda, usrTOL, alpha, betaTV, nprisms, xminPrisms, xmaxPrisms, prismDensity,`

Sequência de execução:

1. Deriva `prismWidth = (xmaxPrisms - xminPrisms) / nprisms`.
2. Se `noise_sigma > 0`, adiciona ruído gaussiano **multiplicativo** — `gz + gz*ruído`, isto é, proporcional à amplitude local, não absoluto. A cópia defensiva de `anomObs` preserva o vetor do chamador.
3. Gera o modelo inicial com `createInitialPrisms` e sua resposta com `functn`.
4. Inicializa `state` com $H_0 = I$.
5. Itera `QNiterate`, imprimindo objetivo, RMS e TV a cada passo.

Critérios de parada

Condição	Teste	Significado
Marquardt esgotado	`flambda < 0`	Nenhum $\lambda \leq 10^8$ produziu redução — mínimo local ou modelo mal-condicionado
Convergência	`conv ≤ usrTOL`	Variação relativa do objetivo abaixo da tolerância por **5 iterações consecutivas**
Limite de iterações	`it == maxIter`	Salvaguarda contra laço infinito

A exigência de cinco iterações consecutivas — e não uma — evita parada prematura em platôs, situação comum quando o termo TV domina temporariamente o funcional.

Bloco de demonstração

Executado com `python python/tv1d/inv_1d.py`, o bloco `__main__` realiza um teste sintético completo: constrói um modelo verdadeiro com três blocos de profundidades 20, 10 e 30 km sobre um perfil de 200 km discretizado em 100 prismas, calcula a anomalia exata, contamina-a com ruído de aproximadamente 1,25 % e inverte. Ao final compara graficamente o modelo recuperado com o verdadeiro.

Desde esta revisão o bloco começa com `np.random.seed(7)`. Antes disso o ruído era sorteado sem semente e cada execução produzia um modelo diferente, o que impedia citar um resultado numérico estável. Com a semente fixa, os valores e as figuras da seção 8 são reproduzidos exatamente ao rodar o script.

6. Visualização

plotter.py 97 linhas · Matplotlib

Duas funções de figura, ambas com layout de dois painéis empilhados e eixo de profundidade invertido — a convenção geofísica em que a profundidade cresce para baixo.

```
plot_model_and_anomaly(x_obs, Dgrav, x_prisms, m)
```

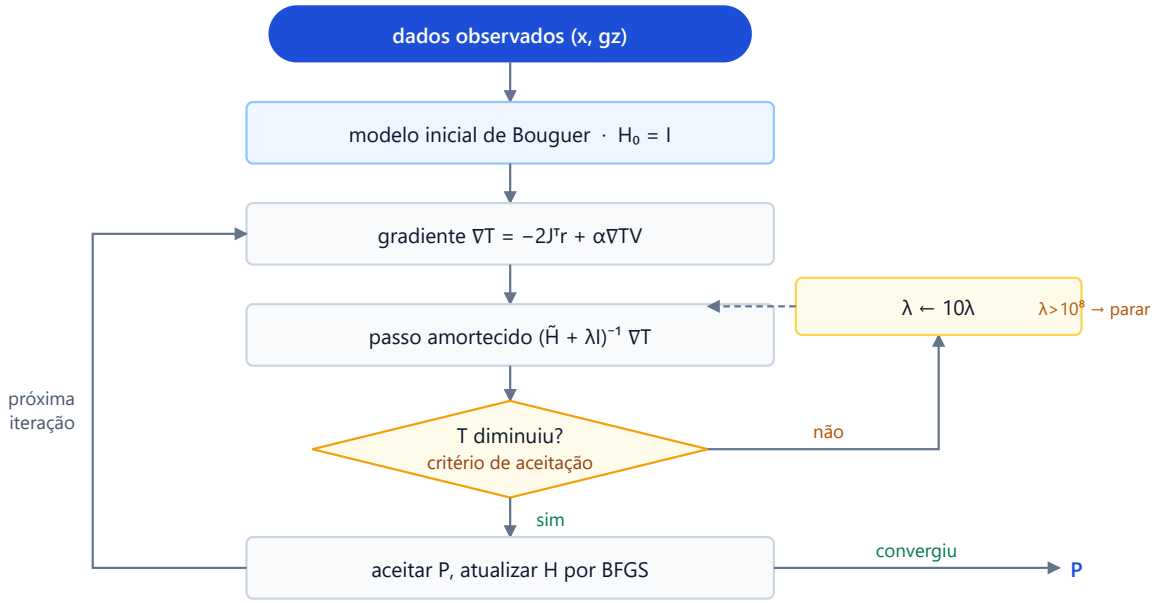
```
plot_inversion_comparison(x_obs, grav_obs, grav_calc, x_prisms_true, model_true, x_prisms_inv, model_inv
```

O uso de `ax.step(..., where='post')` é a escolha correta para esta parametrização: cada prisma tem espessura constante em toda a sua largura, e o gráfico em degraus representa essa descontinuidade fielmente — uma linha interpolada sugeriria uma suavidade que o modelo não possui. O preenchimento com `fill_between` e transparência permite sobrepor os dois modelos sem que um oculte o outro.

Ambas terminam em `plt.show()`, bloqueante em sessão interativa. Para uso em pipelines automatizados, defina o backend não interativo antes da importação (`matplotlib.use('Agg')`) e salve com `savefig`.

7. Fluxo de execução e parâmetros

7.1 Ciclo da inversão



7.2 Parâmetros de controle

Parâmetro	Padrão	Efeito de aumentar o valor
`alpha`	`1.0`	Mais peso à Variação Total: modelo mais achatado e constante por partes, ajuste aos dados pior. É o parâmetro de *trade-off* a ser calibrado — por curva L ou critério de discrepância.
`betaTV`	`0.001`	Aproximação mais suave da norma L_1: converge com mais facilidade, mas borra as descontinuidades que a TV existe para preservar.
`nprisms`	`100`	Discretização mais fina do relevo. Custo cresce com n^3 (inversão de matriz), e o problema fica mais subdeterminado.
`flambda`	`0.001`	Passos iniciais menores e mais conservadores; convergência mais lenta porém mais estável.
`usrTOL`	`0.0001`	Tolerância mais frouxa: para antes, com risco de modelo subconvergido.
`maxIter`	`400`	Teto de iterações.
`prismDensity`	`-0.3`	Contraste de densidade em g/cm³. Negativo para bacia sedimentar sobre embasamento denso. Entra linearmente na equação (1) e é **fortemente correlacionado** com a espessura: erro na densidade se traduz em erro de escala na profundidade.
`udist`	`0`	Unidade de distância: 0 = km (resultado em mGal), 1 = m (conversão de µGal para mGal).
`dcType`	`0`	0 = contraste constante. O valor 1 (variação hiperbólica com a profundidade) está previsto na assinatura mas levanta `NotImplementedError`.
`noise_sigma`	`0.0`	Desvio-padrão do ruído gaussiano multiplicativo aplicado aos dados de entrada. Use apenas em testes sintéticos.

8. Resultado numérico de referência

Execução do bloco de demonstração de `inv_1d.py` com semente fixa, para verificar o comportamento do código após a simplificação do repositório.

Configuração	Valor
Modelo verdadeiro	Três blocos: 20 km (20–60 km), 10 km (80–100 km), 30 km (120–180 km)
Discretização	100 prismas de 2 km ao longo de 200 km
Observações	51 pontos igualmente espaçados
Contraste de densidade	$-0{,}3$ g/cm³
Ruído	Gaussiano multiplicativo, $\sigma = 1/80$ ($\approx 1{,}25$ %)
Regularização	$\alpha = 1{,}0 \cdot \beta = 0{,}001$
Iterações executadas	93

RMS INICIAL

27,4724

mGal, modelo de Bouguer

RMS FINAL

1,9434

mGal, modelo invertido

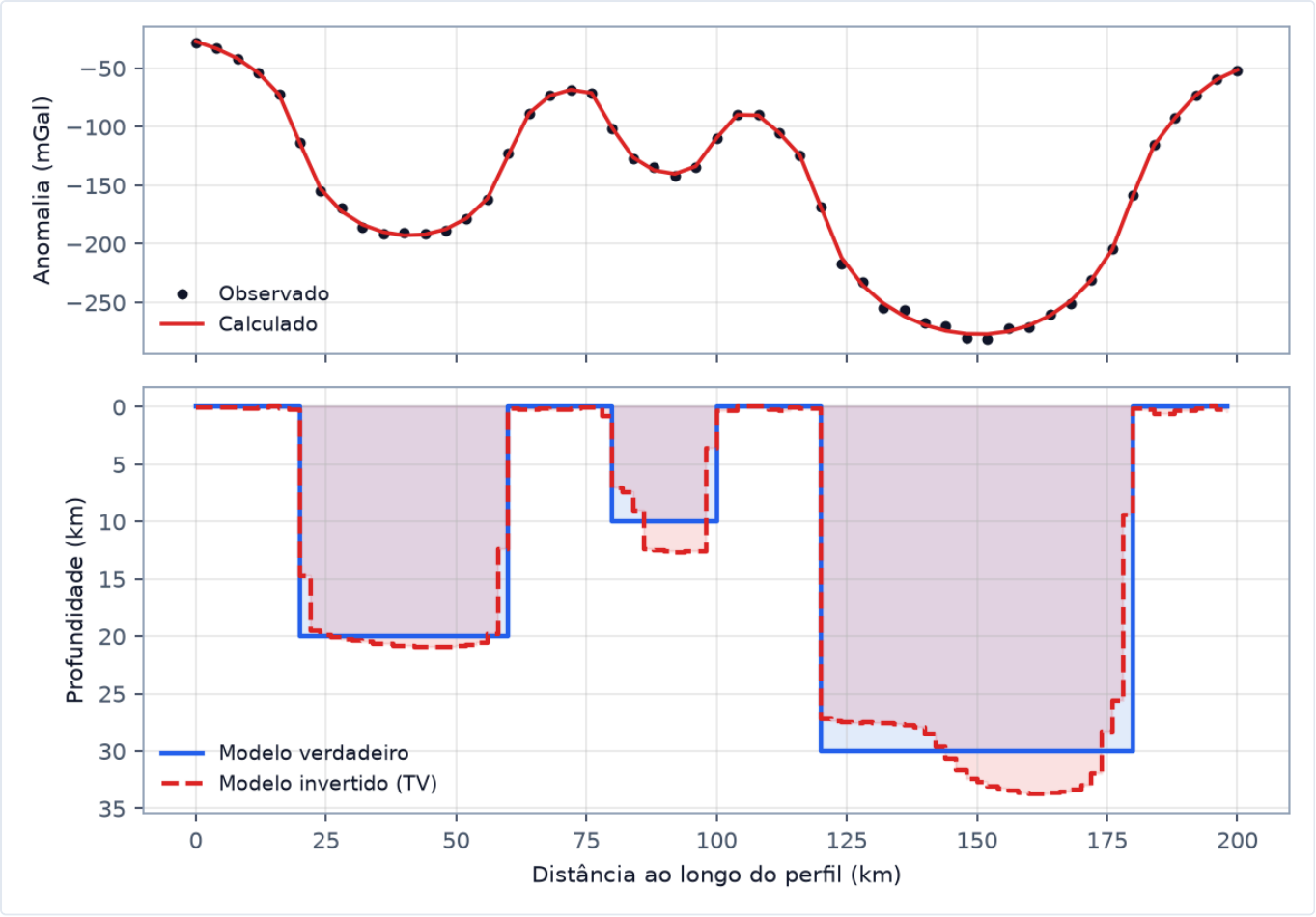


Figura 2. Resultado da inversão. Acima, o ajuste aos dados: os pontos observados (com ruído) e a curva reproduzida pelo modelo invertido, praticamente coincidentes. Abaixo, o modelo verdadeiro em azul contínuo e o recuperado em vermelho tracejado. As **bordas verticais dos três blocos são recuperadas na posição correta e permanecem abruptas** — é exatamente o efeito que a regularização por Variação Total existe para produzir, e que a norma L_2 de suavidade arredondaria. As profundidades são levemente superestimadas (cerca de 34 km contra 30 km no bloco profundo e 12 km contra 10 km no raso) e o interior dos blocos apresenta ondulação suave, comportamento esperado para $\alpha = 1{,}0$ com ruído de 1,25 %.

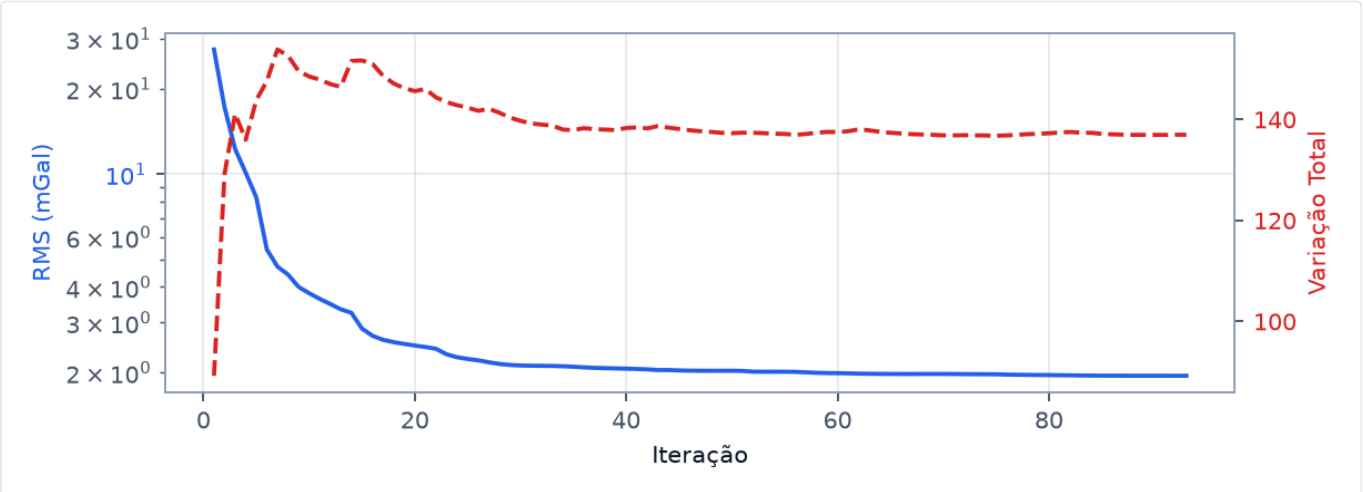


Figura 3. Convergência. Em azul (escala logarítmica, eixo esquerdo) o RMS do ajuste; em vermelho tracejado (eixo direito) o funcional de Variação Total. As duas curvas contam histórias complementares. O modelo inicial de Bouguer é suave — TV baixa — mas ajusta mal os dados. Nas primeiras iterações o otimizador **constrói** a estrutura em blocos: o RMS despenca de 27 para cerca de 4 mGal enquanto a TV *sobe* de 89 até um pico próximo de 155, pois criar degraus custa variação total. A partir da iteração 15, a TV recua e se estabiliza em torno de 137 — o modelo achata o interior dos blocos e concentra a variação nas bordas — enquanto o RMS continua caindo lentamente até 1,94 mGal.

Reprodutibilidade

Todos os números e as duas figuras desta seção saem de `python python/tv1d/inv_1d.py`, que desde esta revisão fixa `np.random.seed(7)` no início do bloco de demonstração. O experimento é, portanto, reproduzível: rodar o script devolve exatamente as 93 iterações e o RMS final de 1,9434 mGal relatados aqui.

9. Correspondência MATLAB → Python

O código Python é um porte do MATLAB preservado em `TVGRAV1D/` e `matlab/`. A tabela abaixo mapeia os arquivos e registra as diferenças deliberadas.

MATLAB	Python	Diferença introduzida no porte
`TVGRAV1D/inv_1D.m`	`tv1d/inv_1d.py`	Leitura de arquivo `input.nl` substituída por argumentos nomeados com valores padrão
`TVGRAV1D/QNiterate.m`	`tv1d/QNiterate.py`	`persistent Hk, bbeta` → dicionário `state` explícito
`TVGRAV1D/functn.m`	`tv1d/functn.py`	Ramo `dcType = 1` convertido em `NotImplementedError`
`TVGRAV1D/functnTV.m`	`tv1d/functnTV.py`	Equivalente
`TVGRAV1D/gradFunctnTV.m`	`tv1d/gradFunctnTV.py`	Equivalente
`TVGRAV1D/fchisq.m`	`tv1d/fchisq.py`	Equivalente
`TVGRAV1D/fderiv.m`	`tv1d/fderiv.py`	Índice `i` passa a ser 0-based
`TVGRAV1D/grav2DPrism.m`	`tv1d/grav2DPrism.py`	`atan` → `np.arctan2`, estável em todos os quadrantes
`TVGRAV1D/dgrav2DPrism.m`	`tv1d/dgrav2DPrism.py`	Equivalente
`TVGRAV1D/createInitialPrisms.m`	`tv1d/createInitialPrisms.py`	Equivalente
`TVGRAV1D/matinv.m`	`tv1d/matinv.py`	Equivalente; nota sobre `np.linalg.inv` no docstring
`matlab/grav2DPrism2.m`	`grav2DPrism2.py`	Equivalente
`matlab/modGrav1d2.m`	`modGrav1d2.py`	**Divergência deliberada:** parâmetro `unit` reintroduzido (ver seção 3). `unit=1` reproduz o MATLAB bit a bit
`matlab/modGrav1dTOTAL.m`	`modGrav1dTOTAL.py`	Repassa `unit` adiante
`matlab/Mod2DGravim.m`	`Mod2DGravim.py`	Aceita `unit`; acrescido de bloco `__main__` de demonstração

9.1 Armadilhas de índice

A conversão de índices 1-based para 0-based foi tratada de forma consistente. O ponto mais sensível é o cálculo do centro do prisma: no MATLAB, `xc = xminPrisms + (i-1)*prismWidth + prismWidth/2` com `i` de 1 a n, em Python, `xc = xminPrisms + i*prismWidth + prismWidth/2` com `i` de 0 a $n-1$. As duas expressões produzem exatamente a mesma sequência de centros.

Em `matinv.py`, os vetores de permutação `IK` e `JK` e as comparações do tipo `(I - K) > 0` foram mantidos na forma original. São corretos porque a comparação é relativa entre índices, invariante ao deslocamento da base.

9.2 O que *não* foi portado

- `hypGrav2DPrism.m` e `dhypGrav2DPrism.m` — contraste de densidade hiperbólico (`dcType = 1`). Os pontos de extensão existem nas assinaturas Python.
- `noise.m` — substituído pelo parâmetro `noise_sigma` de `inv_1d`.
- `plotarmodelo2D.m` e `exemplo1.m` — substituídos por `plotter.py` e pelos blocos `__main__`.
- Leitura dos arquivos `input.nl` e `anomobs.txt` de `TVGRAV1D/Dados/`. Os dados reais e sintéticos permanecem no repositório mas ainda não têm carregador em Python — é a lacuna funcional mais relevante do porte.

10. Pontos de atenção e melhorias sugeridas

10.1 Corretude

Os quatro itens levantados pela análise foram corrigidos nesta revisão. Nenhum deles alterou o resultado numérico da inversão com Variação Total: três eram de clareza e robustez, e o quarto — a unidade da modelagem direta — afetava apenas a cadeia da raiz, não o pacote `tv1d`.

Item	Situação	Descrição e encaminhamento
`modGrav1d2.py`	**Corrigido**	Divisão por 1000 aplicada sem verificar a unidade, deixando a saída fora de mGal. Parâmetro `unit` reintroduzido nas três funções da cadeia, com o mesmo contrato de `tv1d/functn.py`. Detalhes e verificação na seção 3.
`Mod2DGravim.py`	**Corrigido**	`nprisms`, `nobs` e `xmaxPrisms` declarados mas não usados; a geometria efetiva vinha de `len(m)`. As constantes passaram a governar a geometria e um modelo de comprimento incompatível levanta `ValueError`.
`inv_1d.py`	**Corrigido**	O bloco de demonstração não fixava a semente aleatória e execuções sucessivas davam resultados diferentes. Passou a fixar `np.random.seed(7)`.
`QNiterate.py`	**Corrigido**	Variável `idfp` atribuída e nunca lida — resquício da alternância BFGS/DFP. Removida.

10.2 Desempenho

O perfil de custo por iteração é dominado, nesta ordem:

1. **`matinv`** — $O(n^3)$ em laços Python puros. Com $n = 100$ são cerca de 10^6 operações escalares interpretadas por iteração. Substituir por `np.linalg.solve` — que resolve o sistema sem formar a inversa — daria uma aceleração de duas a três ordens de grandeza. A rotina manual pode ser mantida sob uma *flag*, para validação contra o MATLAB.
2. **`functn` e `fderiv`** — $O(n \times n_{\text{obs}})$ chamadas escalares. Como `grav2DPrism` já usa NumPy, basta passar `XOBS` inteiro como vetor e eliminar o laço interno, tornando cada avaliação uma única operação vetorizada.
3. **`gradFunctnTV`** — $2n$ avaliações de um funcional $O(n)$. Com a fórmula analítica da seção 4.2, cai para $O(n)$.

10.3 Extensões úteis para o trabalho

- **Carregador de dados.** Uma função que leia `input.nl` e `anomobs.txt` permitiria aplicar a inversão Python diretamente aos casos reais já presentes em `TVGRAV1D/Dados/` — Büyük Menderes, San Jacinto, Steptoe Valley — e aos sintéticos (graben, bacia costeira, duas sub-bacias).
- **Histórico de convergência.** `inv_1d` devolve apenas `P`. Devolver também as séries de RMS e TV, ou o modelo de cada iteração, evita ter de reprocessar a saída impressa para gerar as figuras de convergência.
- **Curva L para calibrar α.** Rodar a inversão para uma faixa de α e traçar TV(P) contra o RMS produz a curva cujo joelho indica o valor de compromisso — a justificativa quantitativa que a escolha de α pede.
- **Teste de regressão.** Com a semente fixa já em vigor, falta apenas transformar o exemplo em verificação automática: um caso sintético menor, uma tolerância declarada e uma comparação do `P` final contra um

resultado de referência. Isso protege o porte durante a otimização de desempenho sugerida em 10.2, que mexe justamente no caminho numérico.

10.4 Estado do repositório após a simplificação

Foram removidos o pacote `python/tv2d/`, o diretório MATLAB `TVGRAV2D/`, a inversão 2D de resistividade elétrica em `matlab/resis-eletrica/` e os auxiliares de inversão 2D em `matlab/` — as matrizes de suavidade de malha `Msuav.m` e `MsuavL.m` e as rotinas de busca de λ por seção áurea `funcGS2D.m`, `funcGSS2D.m`, `funcGS2Dsuav.m`, `gss2D_lm.m` e `gss2D_lmsuav.m`, que não eram chamadas por nenhum script ativo.

Nenhum módulo remanescente dependia do que foi removido: o pacote `tv1d` nunca importou `tv2d`, e `InversaoRelevoGravimetriaML.m` chama apenas `Mod2DGravim` e `jacobian2DGrav`, ambos preservados. O código Python restante — 16 módulos, 885 linhas — trata exclusivamente do problema 1D.

Na mesma revisão foram aplicadas as quatro correções da seção 10.1: a unidade da cadeia de modelagem direta (parâmetro `unit`, seção 3), a geometria unificada de `Mod2DGravim`, a semente aleatória fixa no exemplo de `inv_1d` e a remoção da variável morta `idfp` em `QNiterate`.

Só a primeira muda um resultado numérico, e apenas na cadeia da raiz. O núcleo da inversão — `functn`, `fderiv`, `fchisq`, `functnTV`, `gradFunctnTV`, `matinv` e a matemática de `QNiterate` — não foi alterado, de modo que as Figuras 2 e 3 permanecem válidas. Os dois pontos de entrada foram reexecutados após as remoções e as correções.
